

Some exercises for preparation for the 1st partial exam, with solutions

Graph Algorithms
UP FAMNIT, 2025/2026
December 3, 2025

1. (From the 1st partial exam, 2024/2025.)

True or false? For each answer provide a brief justification.

- (a) There are two variants of Menger's theorem for digraphs, depending on whether the edges are weighted or not.

Solution: False. There are indeed two variants of Menger's theorem for digraphs, but there are no edge weights. The two variants depend on whether we consider internally vertex-disjoint s, t -paths or edge-disjoint s, t -paths.

- (b) Bellman-Ford algorithm is applicable to any edge-weighted digraph that admits a topological sort.

Solution: True. If a digraph admits a topological sort, then it has no cycles, and in particular it has no negative cycles. Bellman-Ford algorithm is applicable to arbitrary edge-weighted digraphs without negative cycles.

- (c) Suppose that we have a network $N = (D, c, s, t)$ and we would like to compute the minimum capacity of an s, t -cut in N . Then, we can do so in polynomial time by computing a maximum s, t -flow in N using the Ford-Fulkerson algorithm with the improvement of Edmonds and Karp.

Solution: True. In the last step of the Ford-Fulkerson algorithm with the improvement of Edmonds and Karp, we will also compute the set V_f of vertices reachable from s via f -augmenting paths, and the set $\delta^+(V_f)$ will give a minimum capacity s, t -cut in N .

- (d) Every tree is a bipartite graph.

Solution: True. Trees are connected graphs with no cycles, and in particular, trees do not contain odd cycles. Since a graph is bipartite if and only if it has no odd cycles, every tree is a bipartite graph.

2. (From the 1st partial exam, 2024/2025.)

Develop a polynomial-time algorithm for the following problem:

INDEPENDENT SET IN TREES

Input: A tree $T = (V, E)$, an integer k .

Question: Is there an independent set S in T such that $|S| \geq k$?

Hint: Show that the problem admits a polynomial reduction to one of the algorithmic problems that we solved in class.

Solution: In any graph G , a set S of vertices is an independent set if and only if its complement $V(G) \setminus S$ is a vertex cover. Therefore, to solve INDEPENDENT SET IN TREES for a given tree T and integer k , it suffices to check if T contains a vertex cover with cardinality at most $|V(T)| - k$.

In class, we developed a polynomial-time algorithm for computing a minimum vertex cover of a bipartite graph, using the augmenting path method for computing a maximum matching. In the last step, we also obtain a minimum vertex cover C . Since every tree is a bipartite graph, we can run this algorithm on T and conclude that T has a vertex cover with cardinality at most $|V(T)| - k$ if and only if $|C| \leq |V(T)| - k$.

The algorithm solving the INDEPENDENT SET IN TREES problem is as follows:

1. Compute a minimum vertex cover C in T .
 2. If $|C| \leq |V(T)| - k$, then return YES (there is an independent set in T with cardinality at least k).
- Otherwise, return NO (there is no independent set in T with cardinality at least k).

3. (From the 1st partial exam, 2023/2024.)

True or false? For each answer provide a brief justification.

- (a) One of the variants of Menger's theorem gives a characterization of minimum spanning trees.

Solution: False. All the variants of Menger's theorem are about the maximum number of (internally or edge) disjoint paths between two vertices s and t in a graph or a digraph.

- (b) Suppose we have a digraph $D = (V, E)$, real weights $w(e) \in \mathbb{R}$ on the edges, and a vertex s , and we would like to compute a tree of shortest paths from s to all vertices reachable from s . Then, which algorithm applies depends on the properties of the edge-weight function w and on the structure of D .

Solution: True. For example, if D is acyclic, then we can use a dynamic programming approach based on topological sort. If the edge weights are all non-negative, then we can use Dijkstra's algorithm. And if D has no negative cycles, then we can apply Bellman-Ford algorithm. In general, the problem is NP-hard.

- (c) Suppose now that we have a connected undirected graph $G = (V, E)$, positive weights $w(e)$ on the edges of G , and a vertex s . Then, we compute a tree of shortest paths from s to all other vertices using Dijkstra's algorithm.

Solution: True. We can consider G as the directed graph which for each edge $e = uv \in E$ contains a pair of oppositely oriented directed edges (u, v) and (v, u) with the same weight as the weight of e . Since all the edge weights are positive, the algorithm can be applied.

- (d) Let G be a connected graph and let s and t be two vertices in G . Let us perform depth-first search starting in s and let T be the resulting tree. Then, the number of edges on a shortest s, t -path in G is the same as in T .

Solution: False. This would be true for a BFS tree, but not necessarily for a DFS tree. For example, if G is the n -vertex cycle, s and t are two adjacent vertices in G , and the depth-first search starting in s first traverses the edge from s to its neighbor other than t , then T will be the path with endpoints s and t (and $n - 1$ edges). Therefore, the only path from s to t in the tree T has $n - 1$ edges, while in the graph G the shortest s, t -path has a single edge.

- (e) In every network N the maximum capacity of a flow equals the minimum value of a cut.

Solution: False. In every network N the value of a maximum s, t -flow equals the capacity of a minimum s, t -cut.

- (f) In every network in which all capacities are integer-valued, there exists an integer maximum flow.

Solution: True. This is a part of the Max-Flow Min-Cut Theorem.

4. (From the 1st partial exam, 2023/2024.)

Nina excels in math and computer science. Her teacher knows this and asked her to help develop an efficient algorithm for the following problem.

The Graph Algorithms course is attended by 1st and 2nd year students. For the needs of seminar presentations, the teacher would like to divide the students into as few groups as possible, subject to satisfying the following two conditions:

- (1) *No two students in the same group are in the same year of studies.*
- (2) *Every two students in the same group are friends. (Assume that the friendship relation is symmetric.)*

Nina studied the problem and quickly came to the conclusion that the problem can be solved in polynomial time. Is she correct? Of course, you should ignore the fact that in practice the number of students in class is at most a constant; assume that we can deal with arbitrarily large classes.

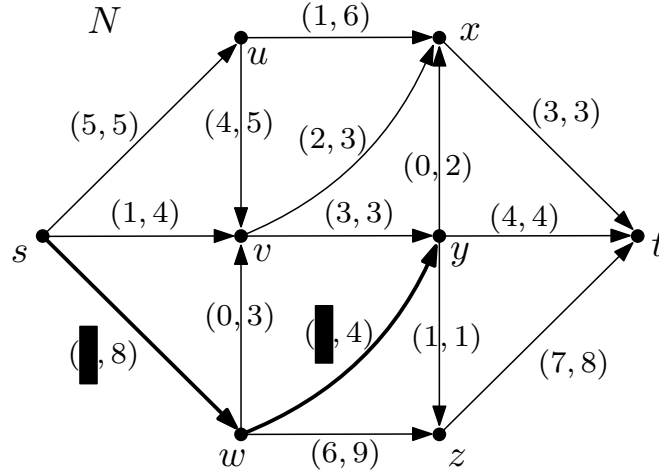
Hint: model the problem as a problem on a bipartite graph.

Solution: Let us introduce the following bipartite graph $G = (V, E)$ with a bipartition $\{V_1, V_2\}$: The vertex set V is the set of students taking the course. The set is partitioned into $V = V_1 \cup V_2$ (disjoint union) where V_i denotes the set of students enrolled in the i -th year, for $i \in \{1, 2\}$. A vertex $x \in V_1$ is adjacent with a vertex $y \in V_2$ if and only if x and y are friends. There are no other edges.

Note that in any valid student grouping, each group of students corresponds to either a single vertex or a pair of adjacent vertices in the graph G . Thus, if k is the number of groups of size two, then the total number of groups is $k + (|V| - 2k) = |V| - k$. This value is minimized precisely when the value of k is maximized. Hence, the problem can be solved in polynomial time by computing a maximum matching in the bipartite graph G .

5. (From the 1st partial exam, 2022/2023.) The figure below depicts a flow f in a network $N = (D = (V, E), c, s, t)$, with ordered pairs $(f(e), c(e))$ on the edges. Some ink has spilled onto the drawing, obscuring the flow values on the boldfaced edges, $e \in \{sw, wy\}$.
- (a) Define the values of $f(sw)$ and $f(wy)$ so that the mapping f is a flow in the network N .

Solution: We apply the flow conservation law for the vertex y and get $f(vy) + f(wy) = f(xy) + f(yz) + f(yt)$
 $3 + f(wy) = 0 + 4 + 1$. It follows that $f(wy) = 2$. Applying the flow conservation law for the vertex w , we get $f(sw) = f(vw) + f(wy) + f(wz) = 0 + 2 + 6 = 8$. Since $0 \leq f(wy) \leq c(wy)$ and $0 \leq f(sw) \leq c(sw)$, the obtained values indeed lead to a flow in the network N .



- (b) Is f a maximum flow? Justify your answer by performing a breadth-first search in the residual network.

Solution: Yes, the flow is maximum. Using breadth-first search from the vertex s in the residual network, we reach the set of vertices $V_f = \{s, u, v, x\}$. Therefore, there is no augmenting path for the flow, so the flow is a maximum flow.

- (c) Find a minimum cut in the network and justify that it is indeed minimum.

Solution: A minimum cut is the set $\delta^+(V_f) = \{(s, w), (v, x), (x, t)\}$. The capacity of the cut is $c(sw) + c(vx) + c(xt) = 8 + 3 + 3 = 14$. This is equal to the value of the maximum flow f , which is $f(su) + f(sv) + f(sw) = 5 + 1 + 8 = 14$. The minimality follows from the Max-Flow-Min-Cut theorem.

6. (Adapted from two problems in the 1st partial exam from 2021/2022.)

- (a) Let $G = (V, E)$ be a connected graph with nonnegative weights on the edges, $w(e) \geq 0$ for all $e \in E$. Assume that you have already computed a tree T of shortest paths to all vertices from a given vertex $s \in V$. Suppose now that each edge weight increases by 1: the new weights are $w'(e) = w(e) + 1$ for all $e \in E$. Does the shortest paths tree change? Give an example of where it changes, or prove that it does not change.

Solution: It can change. For a simple counterexample, we can take a complete graph at three vertices s, t, u and weights on the edges $w(st) = w(tu) = 1, 1$ and $w(su) = 3$. Then (s, t, u) is a shortest path from s to u in the graph G . For the changed weights w' , we have $w'(st) = w'(tu) = 2, 1$ and $w'(su) = 4$. Now (s, t, u) is no longer the shortest path from s to u , the shortest s, u -path becomes the edge (s, u) .

- (b) Let $N = (D = (V, E), c, s, t)$ be an arbitrary network with integer capacities. If f is a maximum s, t -flow in the network N , then $f(e) = c(e)$ holds for all outgoing edges e from the source s .

Solution: False. Example: $V = \{s, t, u\}$, $E = \{(s, t), (s, u)\}$, $c(s, t) = c(s, u) = 1$.

There is a unique maximum flow f , given by $f(s, t) = 1$ and $f(s, u) = 0$.

7. (From one of the exams in 2023/2024.)

Let $D = (V, E)$ be a digraph with positive edge weights $w(e), e \in E$, and let $s, t \in V$. Describe a polynomial-time algorithm to determine whether a shortest s, t -path in D is unique. Analyze its time complexity as a function of $n = |V|$ and $m = |E|$.

Hint: Make a Cook reduction to the shortest path problem with positive edge weights.

Solution: We make a Cook reduction to the shortest path problem with positive edge weights. This problem can be solved by Dijkstra's algorithm in time $\mathcal{O}(n^2)$ (or even $\mathcal{O}(n \log n + m)$). First, we compute a shortest s, t -path P and denote by ℓ its length. (If such a path does not exist, we return NO.) Then, for all $e \in E(P)$, we compute a shortest s, t -path in $D - e$ using Dijkstra's algorithm. We denote by ℓ_e the length of such a path (or ∞ if there is no such path). If $\ell \neq \ell_e$ for all $e \in E(P)$, return YES (since in this case the shortest s, t -path is unique). Otherwise, return NO. Time complexity: $\mathcal{O}(|E(P)|n^2) = \mathcal{O}(n^3)$ (or even $\mathcal{O}(n(n \log n + m))$).

8. (From the list of exercises for Chapter 3, second part.)

Show that there is a Cook reduction between any two of the following problems. In other words, show that the existence of a polynomial-time algorithm for one of them implies the existence of a polynomial-time algorithm for all the others.

- MAXIMUM INDEPENDENT SET: For a given graph G , compute $\alpha(G)$, the maximum size of an independent set in G .
- MINIMUM VERTEX COVER: For a given graph G , compute $\tau(G)$, the minimum size of a vertex cover in G .
- MAXIMUM CLIQUE: For a given graph G , compute $\omega(G)$, the maximum size of a clique in G .
- CLIQUE: Given a graph G and a positive integer k , does G have a clique of cardinality at least k ?

Solution: For every graph G it holds that $\alpha(G) + \tau(G) = |V(G)|$, $\alpha(G) = \omega(\overline{G})$, and $\omega(G) = \alpha(\overline{G})$. With the help of these relations, we can easily establish polynomial equivalence of the problems MAXIMUM INDEPENDENT SET, MINIMUM VERTEX COVER and MAXIMUM CLIQUE, as follows.

- Assume that A_α is a polynomial-time algorithm for the MAXIMUM INDEPENDENT SET problem. Then we can solve the MINIMUM VERTEX COVER problem in polynomial time as follows: for an input graph G we compute $\alpha(G)$ with the algorithm A_α and return $\tau(G) = |V(G)| - \alpha(G)$.

Similarly, if A_τ is a polynomial-time algorithm for the MINIMUM VERTEX COVER problem, then we can solve the MAXIMUM INDEPENDENT SET problem in polynomial time: for an input graph G we compute $\tau(G)$ with the algorithm A_τ and return $\alpha(G) = |V(G)| - \tau(G)$.

So the MAXIMUM INDEPENDENT SET and MINIMUM VERTEX COVER problems are polynomially equivalent.

- Assume that A_α is a polynomial-time algorithm for the MAXIMUM INDEPENDENT SET problem. Then we can solve the MAXIMUM CLIQUE problem in polynomial time, as follows: for an input graph G we compute its complement $\overline{G}$, with the algorithm A_α we compute $\alpha(\overline{G})$, and we return $\omega(G) = \alpha(\overline{G})$.

Similarly, if A_ω is a polynomial-time algorithm for the MAXIMUM CLIQUE problem, then we can solve the MAXIMUM INDEPENDENT SET problem in polynomial time by computing the complement $\overline{G}$, computing its clique number $\omega(\overline{G})$ with the algorithm A_ω , and returning $\alpha(G) = \omega(\overline{G})$.

So the MAXIMUM INDEPENDENT SET and MAXIMUM CLIQUE problems are polynomially equivalent.

- A polynomial equivalence of the MINIMUM VERTEX COVER and MAXIMUM CLIQUE follows from transitivity.

It remains to prove the polynomial equivalence of the MAXIMUM CLIQUE and CLIQUE problems.

Assume first that A_ω is a polynomial-time algorithm for the MAXIMUM CLIQUE problem. Then we can solve the CLIQUE problem in polynomial time, as follows: for an input (G, k) we compute with the algorithm A_ω the clique number $\omega(G)$ and check whether $\omega(G) \geq k$ holds. If the inequality holds, then G has a clique of size at least k , otherwise it does not.

Let us now assume that A'_ω is a polynomial-time algorithm for the CLIQUE problem. Then we can solve the MAXIMUM CLIQUE problem in polynomial time, as follows: for an input graph G we use the algorithm A'_ω for all $k \in \{1, \dots, |V(G)|\}$ to check whether G has a clique of size at least k . The clique number $\omega(G)$ is then the largest $k \in \{1, \dots, |V(G)|\}$ such that G has a clique of size at least k .

9. (From one of the exams in 2022/2023.)

In the shortest path problem, the input is a (directed or undirected) graph $G = (V, E)$, an edge weight function $w : E \rightarrow \mathbb{R}$, and two vertices $s, t \in V$, and the task is to find a shortest s, t -path with respect to the given weight function.

Depending on whether the graph is directed or undirected and on the properties of the weight function, we obtain several problem variants. In the table below several such variants are listed, along with several polynomial-time graph algorithms. For each of problem variant and each algorithm, specify whether the algorithm can be used to solve the problem variant by writing YES / NO in the corresponding table entry. If the input graph is undirected, consider it as a special case of the directed graph in which each edge is replaced with a pair of oppositely oriented arcs.

	BFS	DFS	Dijkstra's algorithm	Bellman-Ford algorithm
G is undirected, unit weights				
G is directed, nonnegative weights				
G is directed, no negative cycles				
G is directed, no restriction on the weights				

Solution:

	BFS	DFS	Dijkstra's algorithm	Bellman-Ford algorithm
G is undirected, unit weights	yes	no	yes	yes
G is directed, nonnegative weights	no	no	yes	yes
G is directed, no negative cycles	no	no	no	yes
G is directed, arbitrary real weights	no	no	no	no

10. (From one of the exams in 2022/2023.)

- (a) Write down the definitions of the following concepts, all related to an undirected graph $G = (V, E)$: independent set / bipartite graph / matching / vertex cover / dominating set.

Solution: Let $G = (V, E)$ be a graph.

An independent set in G is a set of pairwise non-adjacent vertices.

The graph G is bipartite if it admits a bipartition, that is, a partition of V into two independent sets.

A matching in G is a set of pairwise disjoint edges.

A vertex cover in G is a set $S \subseteq V$ such that every edge in G has at least one endpoint in S .

A dominating set in G is a set $S \subseteq V$ such that every vertex in G is either in S or has a neighbor in S . (Equivalently: if every vertex not in S has a neighbor in S .)

- (b) State König-Egerváry Theorem.

Solution: In every bipartite graph the maximum size of a matching equals the minimum size of a vertex cover.

- (c) Show that if a graph G has no isolated vertices, then every vertex cover in G is also a dominating set.

Solution: Let S be a vertex cover and let v be a vertex not in S . Since v is not an isolated vertex, it has a neighbor w . But since S is a vertex cover and $v \notin S$, it must be the case that $w \in S$. Therefore, every vertex not in S has a neighbor in S , that is, S is a dominating set.

- (d) Is every dominating set in a graph G necessarily a vertex cover?

Solution: No. For example, if G is any complete graph with at least 3 vertices, then any set consisting of a single vertex forms a dominating set but not a vertex cover.